

LABORATÓRIO 4 - Sistemas Distribuídos

GRUPO: G11

ALUNOS: Sara Freire, Miguel Medeiros, Luís Pacheco

Repositório GitHub (LAB4): <https://github.com/xaviluisxavier/LAB4>

PARTE A: Introdução do Socket UDP

B1 (b) Explique com detalhe o que acontece com o novo tipo de mensagem UDP_PORT. Por que razão se chama o `lista_clientes.adicionar()` no método `run()`?

O novo comando UDP_PORT serve para o cliente informar o servidor sobre qual é a porta local que o seu Sistema Operativo lhe atribuiu para receber mensagens UDP de *broadcast*. A função `lista_clientes.adicionar()` passou a ser chamada dentro do método `run()` (e já não no momento do `accept()` na classe *Maquina*) porque o servidor apenas tem conhecimento de qual é a porta UDP de destino depois de receber e processar esta mensagem de configuração enviada pela ligação TCP.

B2 (b) Explique que alterações existem na função `adicionar`. Que informação adicional passa a ser guardada?

A função `adicionar` deixou de guardar apenas o objeto de *connection* (TCP). Agora, a estrutura de dados é um dicionário complexo que guarda para cada cliente: a ligação TCP (*connection*), a porta UDP local do cliente (`udp_port`) e um tuplo `udp_address` que contém o IP e a porta combinados para facilitar o posterior envio de datagramas.

B2 (c) Qual deverá ser o papel no código da nova função `obter_destinos_udp()`?

Esta função serve como um filtro para a *Thread* de *broadcast*. Ela percorre o dicionário global de clientes conectados e extrai exclusivamente os tuplos `udp_address` (IP e porta UDP) de cada um, isolando a informação de roteamento necessária para o envio do histórico.

B3 (b) Explique o que faz a nova função `broadcast_object()`, em particular o que faz o ciclo `for`. Se existirem 2 clientes ligados, quantas mensagens são enviadas?

A função serializa o objeto do histórico para JSON e utiliza o `socket` UDP para o enviar. O ciclo `for` itera sobre a lista de destinos obtida em `obter_destinos_udp()`, extraíndo o endereço exato de cada cliente e executando um `sendto`. Se existirem 2 clientes ligados, o ciclo é percorrido duas vezes, sendo enviadas exatamente 2 mensagens de datagrama de forma individual.

B3 (c) Apresente as diferenças entre a forma como a comunicação é feita através deste novo `socket` UDP e o anterior TCP/IP. * `Socket` TCP/IP:

É orientado à conexão. Utiliza o `accept()` para criar um canal dedicado (a variável `connection`) por onde os dados fluem como um fluxo contínuo.

- **Socket UDP:** Não estabelece um canal fixo. Funciona através do envio de datagramas independentes, obrigando a que cada chamada de envio (`sendto`) especifique explicitamente o endereço de destino (IP e porta) daquele pacote específico.

A2 (b) A mensagem recebida apresenta o histórico. Explique que dados estão no histórico.

O histórico contém um dicionário onde as chaves são os tipos de operações matemáticas ('soma', 'subtrair', 'dividir'). O valor de cada chave é uma lista de registos. Cada registo contém os operadores escolhidos (X e Y), o resultado da operação, o IP/porta do cliente que fez o pedido, e a *timestamp* (hora exata do cálculo).

A2 (c) Explique se esta classe cria uma nova `Thread`... Apresente a estrutura global.

Sim, a classe `BroadcastReceiver` herda de `threading.Thread`, criando uma linha de execução secundária no cliente focada apenas em escutar a porta UDP, sem bloquear a interação do utilizador.

- **Estrutura do Servidor:** 1 *Thread* principal (aceita conexões TCP) + 1 *Thread* `ThreadBroadcast` (envia os históricos por UDP a cada 10s) + *N Threads* `ProcessaCliente` (uma para gerir os cálculos TCP de cada cliente conectado).
- **Estrutura do Cliente:** 1 *Thread* principal (gere a interface, lê *inputs* e comunica cálculos via TCP) + 1 *Thread* `BroadcastReceiver` (bloqueada na receção de datagramas UDP).

A3 (a) Identifique quais as linhas de código introduzidas e qual é a sua função.

```
def __init__(self):
    # Ligação TCP para pedidos/respostas da calculadora
    self.connection = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.connection.connect((cliente.SERVER_ADDRESS, cliente.PORT))

    # Socket UDP dedicado para receber broadcasts
    self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    self.udp_socket.bind(('', 0)) # porta livre atribuída pelo SO
    self.udp_port = self.udp_socket.getsockname()[1]

    # Informa o servidor de que a seguir será enviado o porto UDP
    self.send_str(self.connection, cliente.UDP_PORT)
    self.send_int(self.connection, self.udp_port, cliente.INT_SIZE)
    print(f"Cliente ligado por TCP; à escuta de broadcasts UDP na porta {self.udp_port}")
    broadcast = BroadcastReceiver(self.udp_socket)
    broadcast.start()
```

- **Função:** Instanciar a classe passando-lhe o *socket* UDP já configurado, e chamar o método `.start()` para iniciar a nova *Thread*.

Validação de Conhecimentos (Parte A)

- **[X] O servidor tem uma Thread principal... O cliente tem duas Thread...**
 - **Justificação:** O servidor aceita clientes e delega-os para novas *Threads*. O cliente usa a principal para a interface e uma secundária isolada para receber os datagramas de histórico por UDP.
- **[X] É o comando que chama o método `run()` da Thread. Se a Thread tem a designação de processo então o comando é `process.start()`**
 - **Justificação:** O método `.start()` é o mecanismo nativo do Python para lançar efetivamente a *Thread* em paralelo, que chama internamente o `.run()`.
- **[X] O socket UDP não estabelece um canal de comunicação...**
 - **Justificação:** O UDP não usa `accept()`, exigindo que o destino seja explicitado a cada envio, ao contrário do TCP que retém a informação no objeto `connection`.
- **[X] Diferenciar o canal de comunicação para as mensagens...**
 - **Justificação:** Separa o canal crítico de pedidos/respostas síncronas (TCP) do canal de atualizações assíncronas de estado (UDP), evitando o cruzamento de mensagens no mesmo canal.

PARTE B: Interface Gráfica com Pygame

1 (a) Registe as questões colocadas e respostas colocadas no documento a entregar.

> **Utilizador:** "Cria uma interface gráfica para a minha calculadora usando Pygame. Mantém o ficheiro interface original intacto. O Pygame deve captar eventos de teclado (números e sinais) e desenhar no ecrã."

IA Generativa: "Para o Pygame, precisará de uma classe com um método inicializador de rede igual à interface antiga e um método principal com um ciclo *while running*."

1 (b) Explique como funciona o ciclo principal de execução do PyGame.

O ciclo principal do PyGame funciona num laço *while* infinito (controlado por uma *flag* booleana *running*). A cada iteração deste ciclo (normalmente regulada por um "clock" para definir os *frames per second*), o programa:

1. Lê os eventos do utilizador (`pygame.event.get()`);
2. Atualiza o estado interno (variáveis como `input_x` e `input_y`);
3. Redesenha o ecrã limpando o fundo e "pintando" os novos textos por cima utilizando a função `pygame.display.flip()`.

1 (c) Explique as alterações que foi propondo ao chat de forma a obter o protótipo final.

Pedi à IA para não colocar caixas de texto complexas que exigissem cliques com o rato, mas sim implementar a alternância entre a variável X e Y utilizando a tecla TAB. Adicionalmente, solicitei que a comunicação com o *socket* só acontecesse quando as teclas de operação (+, -, /) fossem pressionadas, de modo a impedir que a interface bloqueasse precocemente à espera de uma resposta do servidor.

Validação de Conhecimentos (Parte B)

1. Pygame

- **[X] O Pygame não inicia uma nova Thread quando arranca...**
 - **Justificação:** O Pygame é executado de forma sequencial na *Thread* principal. A ilusão de fluidez e simultaneidade vem da rapidez com que o laço *while* processa os eventos do teclado/rato e redesenha o monitor, não dependendo do lançamento de rotinas *multithreading* na sua base.

2. Interação com o utilizador

- **[X] O Pygame permite aceder às teclas pressionadas utilizando código preparado para o efeito.**
 - **Justificação:** Através da verificação do evento `pygame.KEYDOWN` no laço de eventos, é possível detetar qual a tecla física ou o caractere Unicode correspondente inserido pelo utilizador.